

SSF Formulations

Scott Forrest

2025-09-15

In this script we demonstrate different approaches to including covariates in an SSF model.

Table of contents

Load required packages	1
Import data and clean	1
Create a step object	3
Plot the data spatially	3
Pick out a single individual	4
Create steps	5
Import spatial covariates	6
Extract covariate values at the end of each step	7
Prepare data for extracting covariate information	8
References	8

Load required packages

```
library(tidyverse)
packages <- c("amt", "sf", "terra", "RColorBrewer")
walk(packages, require, character.only = T)
```

Import data and clean

```
buffalo_data <- read_csv("data/buffalo.csv")
```

New names:

Rows: 133161 Columns: 11

-- Column specification

----- Delimiter: "," chr
(2): node, dates dbl (7): ...1, lat, lon, height, accuracy, heading, speed dtm
(2): timestamp, DateTime
i Use `spec()` to retrieve the full column specification for this data. i
Specify the column types or set `show\_col\_types = FALSE` to quiet this message.
* `` -> `...1`

```
# remove individuals that have poor data quality or less than about 3 months of data.  
# The "2014.GPS_COMPACT copy.csv" string is a duplicate of ID 2024, so we exclude it  
buffalo_data <- buffalo_data %>% filter(!node %in% c("2014.GPS_COMPACT copy.csv",
```

```
2029, 2043, 2265, 2284, 2346))
```

```
buffalo_data <- buffalo_data %>%  
  group_by(node) %>%  
  arrange(DateTime, .by_group = T) %>%  
  distinct(DateTime, .keep_all = T) %>%  
  arrange(node) %>%  
  mutate(ID = node)
```

```
buffalo_clean <- buffalo_data[, c(12, 2, 4, 3)]  
colnames(buffalo_clean) <- c("id", "time", "lon", "lat")  
attr(buffalo_clean$time, "tzzone") <- "Australia/Queensland"  
head(buffalo_clean)
```

A tibble: 6 x 4

	id	time	lon	lat
	<chr>	<dtm>	<dbl>	<dbl>
1	2005	2018-07-25 10:04:02	134.	-12.3
2	2005	2018-07-25 11:04:23	134.	-12.3
3	2005	2018-07-25 12:04:39	134.	-12.3
4	2005	2018-07-25 13:04:17	134.	-12.3
5	2005	2018-07-25 14:04:39	134.	-12.3
6	2005	2018-07-25 15:04:27	134.	-12.3

```
tz(buffalo_clean$time)
```

```
[1] "Australia/Queensland"
```

```
buffalo_ids <- unique(buffalo_clean$id)
```

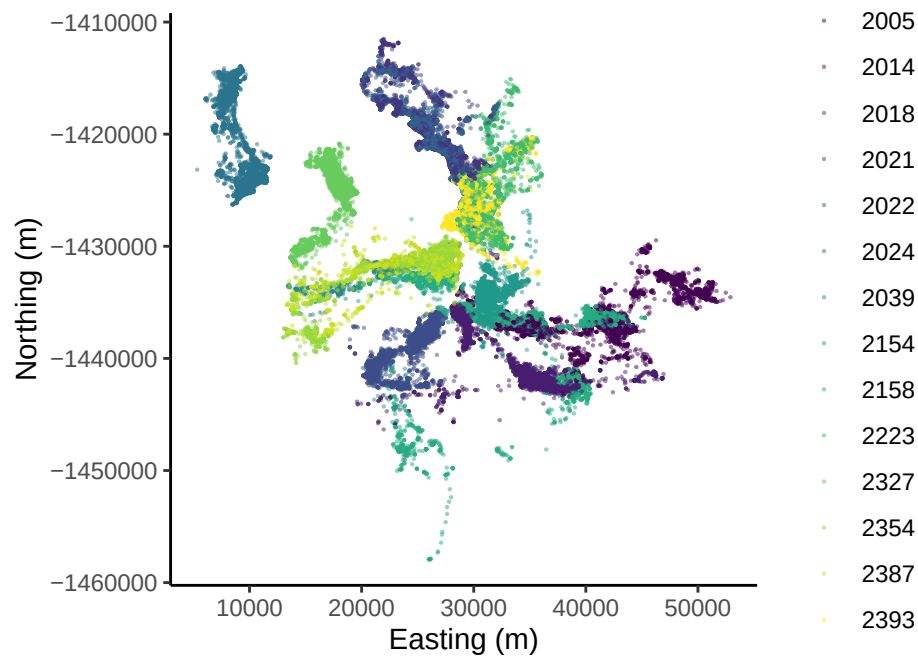
Create a step object

Use the `amt` package to create a trajectory object from the cleaned data.

```
buffalo_all <- buffalo_clean %>% mk_track(id = id,  
                                           lon,  
                                           lat,  
                                           time,  
                                           all_cols = T,  
                                           crs = 4326) %>%  
  transform_coords(crs_to = 3112, crs_from = 4326) # Transformation to GDA94 /  
# Geoscience Australia Lambert (https://epsg.io/3112)
```

Plot the data spatially

```
buffalo_all %>%  
  ggplot(aes(x = x_, y = y_, colour = id)) +  
  geom_point(alpha = 0.5, size = 0.1) +  
  coord_fixed() +  
  scale_x_continuous("Easting (m)") +  
  scale_y_continuous("Northing (m)") +  
  scale_colour_viridis_d() +  
  theme_classic() +  
  theme(legend.position = "right")
```



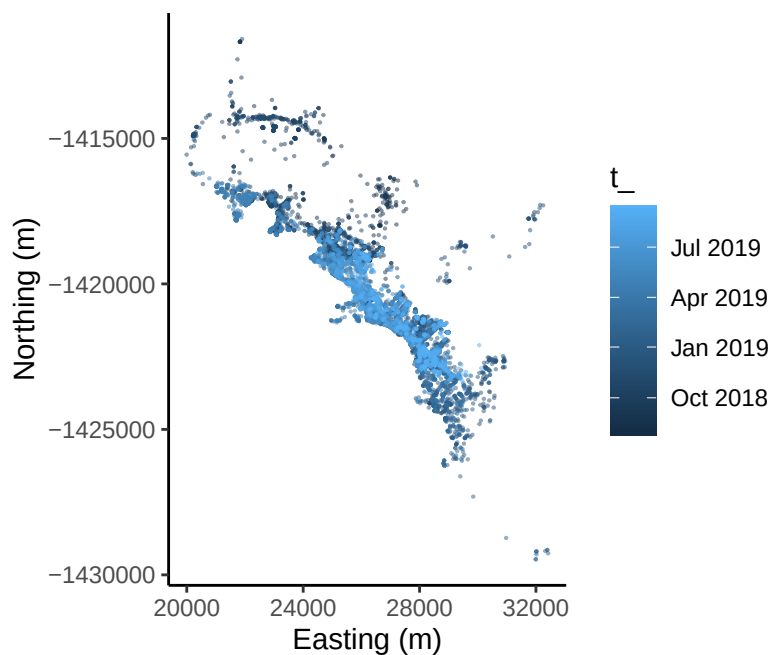
```
# ggsave("outputs/data_prep/buffalo_djelk_map.png",
#         width = 150, height = 150, units = "mm", dpi = 600)
```

Pick out a single individual

```
which_buffalo <- "2022" # select a single buffalo ID

buffalo_id <- buffalo_all %>% filter(id == which_buffalo)

buffalo_id %>%
  ggplot(aes(x = x_, y = y_, colour = t_)) +
  geom_point(alpha = 0.5, size = 0.1) +
  coord_fixed() +
  scale_x_continuous("Easting (m)") +
  scale_y_continuous("Northing (m)") +
  theme_classic()
```



Create steps

We will just use a steps object here for now

```
buffalo_id_steps <- buffalo_id %>%
  steps()

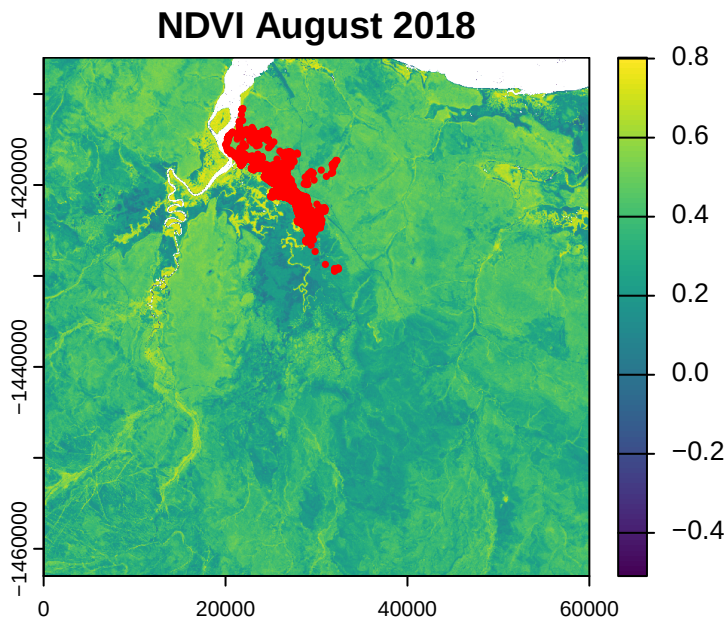
head(buffalo_id_steps)
```

```
# A tibble: 6 x 10
   x1_    x2_    y1_    y2_    sl_ direction_p  ta_ t1_
* <dbl> <dbl>   <dbl>   <dbl> <dbl>   <dbl>   <dbl> <dtm>
1 27929. 27931. -1421151. -1421156.  5.95    -1.29    NA    2018-07-
25 10:35:16
2 27931. 27938. -1421156. -1421157.  7.42    -0.0187  1.27  2018-07-
25 11:36:05
3 27938. 27934. -1421157. -1421155.  3.89     2.80    2.82  2018-07-
25 12:34:52
4 27934. 27938. -1421155. -1421160.  5.98    -0.924   2.56  2018-07-
25 13:34:14
5 27938. 27938. -1421160. -1421157.  3.37     1.52    2.44  2018-07-
25 14:34:08
6 27938. 27940. -1421157. -1421154.  2.92     0.884  -0.635 2018-07-
25 16:34:33
# i 2 more variables: t2_ <dtm>, dt_ <drtn>
```

Import spatial covariates

Although NDVI changes over time, and we have access to monthly layers, we will just select a single month here.

```
ndvi <- rast("mapping/ndvi_aug_2018.tif")
plot(ndvi, main = "NDVI August 2018")
points(buffalo_id$x_, buffalo_id$y_, col = "red", pch = 16, cex = 0.5)
```



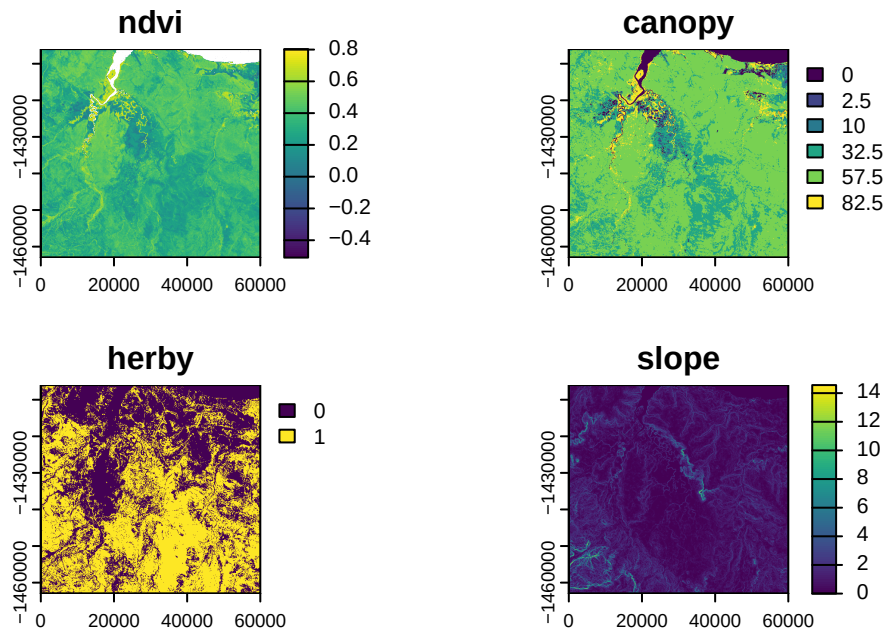
```
ndvi
```

```
class      : SpatRaster
dimensions : 2280, 2400, 1  (nrow, ncol, nlyr)
resolution : 25, 25  (x, y)
extent     : 0, 60000, -1463000, -1406000  (xmin, xmax, ymin, ymax)
coord. ref.: GDA94 / Geoscience Australia Lambert (EPSG:3112)
source     : ndvi_aug_2018.tif
name       :      ndvi
min value  : -0.5441047
max value  :  0.8086554
```

```
canopy <- rast("mapping/canopy_cover.tif")
herby <- rast("mapping/veg_herby.tif")
slope <- rast("mapping/slope_raster.tif")

spatial_covs <- c(ndvi, canopy, herby, slope)
```

```
names(spatial_covs) <- c("ndvi", "canopy", "herby", "slope")
plot(spatial_covs)
```



Extract covariate values at the end of each step

```
buffalo_id_steps <- buffalo_id_steps %>%
  extract_covariates(spatial_covs, where = "end")

head(buffalo_id_steps)
```

```
# A tibble: 6 x 14
  x1_   x2_   y1_   y2_   sl_ direction_p   ta_ t1_
* <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dtm>
1 27929. 27931. -1421151. -1421156.  5.95   -1.29   NA   2018-07-
25 10:35:16
2 27931. 27938. -1421156. -1421157.  7.42   -0.0187  1.27  2018-07-
25 11:36:05
3 27938. 27934. -1421157. -1421155.  3.89    2.80    2.82  2018-07-
25 12:34:52
4 27934. 27938. -1421155. -1421160.  5.98   -0.924   2.56  2018-07-
25 13:34:14
5 27938. 27938. -1421160. -1421157.  3.37    1.52    2.44  2018-07-
25 14:34:08
```

```

6 27938. 27940. -1421157. -1421154. 2.92      0.884 -0.635 2018-07-
25 16:34:33
# i 6 more variables: t2_ <dtm>, dt_ <drtn>, ndvi <dbl>, canopy <dbl>,
#   herby <dbl>, slope <dbl>

```

Prepare data for extracting covariate information

```

buffalo_id_steps <- buffalo_id_steps %>%
  mutate(t1_ = lubridate::with_tz(buffalo_id_steps$t1_, tzzone = "Australia/Darwin"),
         t2_ = lubridate::with_tz(buffalo_id_steps$t2_, tzzone = "Australia/Darwin"))

buffalo_id_steps <- buffalo_id_steps %>%
  mutate(
    # id_num = as.numeric(factor(id)),
    # step_id = step_id_,
    x1 = x1_, x2 = x2_,
    y1 = y1_, y2 = y2_,
    t1 = t1_,
    t1_rounded = round_date(t1_, "hour"),
    hour_t1 = hour(t1_rounded),
    t2 = t2_,
    t2_rounded = round_date(t2_, "hour"),
    hour_t2 = hour(t2_rounded),
    hour = ifelse(hour_t2 == 0, 24, hour_t2),
    yday = yday(t1_),
    year = year(t1_),
    month = month(t1_),
    sl = sl_,
    log_sl = log(sl_),
    ta = ta_,
    cos_ta = cos(ta_),
    canopy_01 = canopy/100)

```

References